

RAG 服务化

L01

Retrieve

Query + Hybrid
改写 / HyDE / Adaptive

L02

Rerank

3 大门派
Cross / ColBERT / LLM

L03

Generate

受约束 + Context
Citations + Prompt Cache

L04

Prompt + Eval

Prompt as Code
LLM-Judge + Bad Case

L05

Release

原子绑定 + Cost
秒级回滚 + 省 75%

把「输入侧」工程化所有成果组装成生产级 RAG 服务 · 可上线 · 可回滚 · 可审计



LESSON 01 · 混合检索

L01 开场 · 纯向量检索为啥在生产里翻车

真实场景 · 订单号检索翻车

前几年我做客服 RAG，团队从「文本 → embedding → 向量库 → 相似度」起步，Demo 跑得溜。上线后客户问「订单号 N-1234-AB 的退款流程」—— 向量检索召回的全是「退款 / 流程 / 客服」相关的语义近邻，正确订单完全没在 top-50 里。那不是模型笨，是「专有名词 / 编号 / 代码」这类「字符精确匹配」场景，纯向量天然干不过 BM25。

咱们这节聊 4 件事：

纯向量 vs 纯 BM25 各自死在什么场景 —— 不是工具不好，是场景不对

Hybrid Search 的 RRF 融合算法 —— 数学上为什么这样设计

pgvector + BM25 + RRF 真实代码 —— OmniSupport 项目能复用

2025 向量库横评 —— Pinecone / Weaviate / Qdrant / Milvus / pgvector 怎么选

WHY HYBRID

纯向量 vs 纯 BM25 · 为什么单一检索必坏

场景	纯向量(Dense)	纯 BM25(Sparse)	Hybrid 必要性
专有名词 / 订单号 / 编号	X 召回不回(语义近邻干扰)	✓ 字面匹配精确	高 ★
同义改写 / 概念相近	✓ 召回得到	X 字面不匹配	高 ★
代码 / 标识符 / 错误码	X 几乎完全失效	✓ 必杀	极高 ★★
长尾语义 / 推理问题	✓ 优势	X 关键词不全	中
短词 query(1-3 字)	差(向量噪声大)	中(频率倒数)	高 ★
多语言 / 跨语种	✓ 跨语种(mxbai-embed-large)	X 死死锁在原语言	高 ★

踩坑提醒

Demo 阶段纯向量看着够用 · 生产里 30-50% query 是「带编号 / 代码 / 错误码」 · 你 RAG 召回率立刻塌一半

HYBRID IS DEFAULT

混合检索 · 2024 后是 RAG 的「默认配置」

原理

向量检索擅长「语义近邻」· BM25 擅长「字面精确」—— 两者天然互补,不是替代关系。
Hybrid Search = 同时跑两路检索 → 用 RRF(Reciprocal Rank Fusion)把两个排序合并 → 出统一 top-k。
关键是 RRF 不需要分数归一化,只看排名,实现极简、效果稳。

工业界验证数据(2024-2025)

方案	NDCG@10	相对提升	来源
纯向量(基线)	基线 100	—	Anthropic 2024.09 blog
纯 BM25	85-95	-5 to -15%	Anthropic / Pinecone
Hybrid (BM25 + Dense + RRF)	115-135	+15-35%	Anthropic 2024 / Pinecone Lab 2025
+ Reranker (Cohere 3.5)	140-160	+40-60%	Cohere 2024.12 / 实测 OmniSupport

4 RETRIEVAL STRATEGIES

4 种主流检索策略 · 从单一到组合

Dense · 纯向量

入门

- embedding 模型相似度
- OpenAI / BGE / Voyage / Cohere
- 优:语义召回强
- 缺:专有名词死
- 适用:对话 / 长尾问题

Sparse · BM25 / SPLADE

经典

- 词频 + 倒文档频率
- Lucene / Elasticsearch / SPLADE
- 优:精确匹配 + 快
- 缺:同义词失效
- 适用:专有名词 / 代码

Hybrid · Dense + BM25

默认 ★

- 两路并发 + RRF 融合
- pgvector + tsvector / Pinecone hybrid
- 优:互补 +15-35%
- 缺:索引双份
- 适用:90% 生产场景

Hybrid + Rerank · 二阶

最强

- Hybrid 召 50 → Reranker 精排 5
- + Cohere 3.5 / Voyage / BGE
- 优:+40-60%
- 缺:贵 + 慢
- 适用:高质量要求场景

RRF · RECIPROCAL RANK FUSION

RRF 算法 · 混合检索的核心融合机制

RRF 公式 · 不需要分数归一化,只看排名

```
def rrf(rankings, k=60):  
    """  
    rankings: list of [doc_id ranked by score]  
    k: 平滑参数 · 工业默认 60  
    """  
    scores = {}  
    for ranking in rankings:      # 各路检索结果  
        for rank, doc in enumerate(ranking, start=1):  
            scores[doc] = scores.get(doc, 0) + 1 / (k + rank)  
    return sorted(scores, key=scores.get, reverse=True)  
  
# 例:  
# dense_top10 = [d1, d5, d3, ...]  
# bm25_top10 = [d3, d7, d1, ...]  
# rrf([dense_top10, bm25_top10]) → 融合排名
```

为什么 k=60?

- Cormack 2009 原始论文实验值
- Pinecone / Vespa 工业实测最稳
- 范围 30-100 影响小

为什么不归一化分数?

- 不同检索器分数尺度差很大
- Cosine 0-1 / BM25 可以 50+
- 排名是「天然归一化」

为什么不用「加权求和」?

- 加权求和需要把 cosine(0-1) 和 BM25(50+) 归一化 · 归一化算法选错就翻车
- 权重 α / β 是超参数 · 每个数据集都得调 · 不可移植
- RRF 只看排名 → 完全不依赖分数尺度 → 跨数据集稳定
- RRF = 工业最稳的「无脑用」融合 · 实在不够再上 learned fusion

REAL ARTIFACT

OmniSupport 项目 · pgvector + BM25 + RRF 真实代码

```
# pipelines/retrieve/hybrid.py
async def hybrid_search(query: str, top_k=50, k=60):
    # 1. 并发跑两路
    dense_task = pg.fetch(
        """SELECT doc_id, 1 - (embedding <=> $1) AS score
        FROM chunks ORDER BY score DESC LIMIT 50""",
        embed(query),
    )
    bm25_task = pg.fetch(
        """SELECT doc_id, ts_rank(tsv, plainto_tsquery($1)) AS score
        FROM chunks WHERE tsv @@ plainto_tsquery($1)
        ORDER BY score DESC LIMIT 50""",
        query,
    )
    dense, bm25 = await asyncio.gather(dense_task, bm25_task)

    # 2. RRF 融合
    scores = defaultdict(float)
    for rank, row in enumerate(dense, 1):
        scores[row.doc_id] += 1 / (k + rank)
    for rank, row in enumerate(bm25, 1):
        scores[row.doc_id] += 1 / (k + rank)

    # 3. 排序 + 取 top_k + 注入 evidence
    ranked = sorted(scores.items(), key=lambda x: -x[1][:top_k])
    return await fetch_chunks_with_evidence(
        [doc_id for doc_id, _ in ranked]
    )
```

并发跑两路 · asyncio.gather

- Dense / BM25 同时跑
- 延迟 = max(两者) 不是 sum

RRF · k=60 工业默认

- defaultdict 累加分数
- 不归一化,直接排名相加

top_k=50 给下一阶段 reranker

- 不要直接喂 5 个给 LLM
- 让 reranker 精排到 3-5

evidence 注入

- bbox / page_no / version 必带
- 直接对接 L03 Citations API

生产小贴士:tsv 字段记得加 GIN 索引

VECTOR DB LANDSCAPE 2025

向量库横评 · 5 个梯队按规模分

< 1M chunk 用 pgvector · 1M-10M 用 Qdrant / Weaviate · 10M+ 上 Milvus / Pinecone

工具	规模	Hybrid 原生	关键特性 2025	推荐场景
pgvector 0.8+	< 1M chunk	需 BM25 自配(tsv)	HNSW + IVFFlat / 整合 Postgres	OmniSupport 默认 / 中小规模
Qdrant 1.13+	1M - 50M	✓ 原生 + RRF	gRPC / 分布式 / 量化	中型项目 / 自托管
Weaviate 1.27+	1M - 100M	✓ 原生 + alpha 融合	GraphQL / 模块化	需要数据契约
Milvus 2.5+	100M+	✓ 原生 + WAND	集群 / 分片 / GPU 加速	超大规模 / 多模态
Pinecone	serverless	✓ Hybrid API	托管 / Assistant / 自动 rerank	不想运维 / 快速上
Elasticsearch 8.15+	海量	✓ KNN + BM25 一体	老牌 + 全文检索成熟	已有 ES 集群 / 复用

INDUSTRY SIGNALS 2025

混合检索行业 4 个明显方向 · 2024-2025 已经收敛

不是「百家争鸣」——是「Hybrid + Rerank + 上下文增强」已经成默契

Anthropic Contextual Retrieval

2024.09

Chunk 加 50-100 token 上下文前缀 + Hybrid + Reranker。
检索失败率 5.7% → 1.9% (-67%) · Anthropic 官方 blog。

Jina Late Chunking

2024.09

先全文 embedding 再切片 · 长文档跨段引用不丢。
Jina-embeddings-v3 8K context 是配套。

Pinecone Hybrid + Assistant

2024-2025

serverless hybrid 一行 API · Assistant 内置 Rerank。
Multi-tenant 友好, 2025 增长最快的托管方案。

Vespa / ColBERT-late

2024-2025

Late interaction 范式 · ColBERT / ColPali 视觉文档。
极大规模 + 复杂 ranking 的工业级答案。



QUERY UNDERSTANDING

检索前的「Query 理解层」· 4 件套补齐 Hybrid 之前的盲区

用户 query 80% 是「烂的」· 先做 query 改写比硬调向量库更有效

真实案例 · 电商客服 RAG 上线 3 周被骂

团队把 Hybrid + Cohere Rerank 都堆上了· recall@5 仍然只有 62%· 客户骂「这破玩意」。
后来加一层 LLM-based Query Rewrite —— recall@5 从 62% 跳到 84%· 没换任何检索组件。
根因· 用户写「那个上周买的多少钱」· embedding 根本不知道「上周买的」是哪个 SKU· 改写后变「订单 #4521 的成交价」直接命中。

- 模糊 → 明确
- LLM 改写口语化 query
- "那个产品多少钱" → "Product X 2026 零售价"
- 场景:对话型 RAG 标配

- 复合 → 子问
- LLM 拆解 multi-fact query
- "A 创始人 + 在哪上学" → ["A 创始人", "X 母校"]
- 场景:Agent 知识库

- Query → Doc
- 让 LLM 假想答案再 embed
- Recall +10-20% (Cohere)
- 场景:垂直专业领域

- 判类型 → 分流
- 事实 / 关系 / 计算 分流
- 事实→向量 关系→图 数→SQL
- 场景:Adaptive RAG 入口

REAL ARTIFACT · REWRITE + HYDE

Query 改写 + HyDE 完整代码 · GPT-4o-mini 跑一次 < \$0.0001

生产实战 · 改写比 ANN 调参便宜 / 见效快 / 不动索引

```
# services/rag_api/query_rewriter.py
from openai import OpenAI
client = OpenAI()

REWRITE_PROMPT = """你是 RAG 查询改写助手。
基于历史对话和最新 query,改写成明确、自包含的检索 query。

历史对话:
{history}

最新 query: {query}

要求:
- 替换代词 (那个/这个) 为具体实体
- 补齐隐含约束 (时间/版本)
- 不要发明事实,只做改写

输出严格 JSON: {"rewritten": "...", "intent": "fact|relation|compute"}
"""

async def rewrite_query(query: str, history: list[dict]):
    resp = await client.chat.completions.create(
        model="gpt-4o-mini-2024-07-18", # 便宜够用
        response_format={"type": "json_object"},
        messages=[{"role": "system",
                    "content": REWRITE_PROMPT.format(
                        history=format_history(history), query=query)}],
    )
    return json.loads(resp.choices[0].message.content)

# HyDE · 让 LLM 假想答案再 embed
async def hyde_embed(query: str):
    hypo = await llm.complete(f"用一段文字回答:{query}")
    return await embedder.embed(hypo) # 用假想答案的向量做 ANN
```

Query Rewrite

- 用 GPT-4o-mini 跑就够
- response_format = json 强制结构化
- 每次 < \$0.0001

HyDE 适用场景

- 垂直领域 (法律/医疗/金融)
- 用户 query 短而模糊
- recall +10-20% (Cohere paper)

何时不要用 HyDE

- 短事实 query
- LLM 假想答案误导性强

何时不要用改写

- 简单关键词 (SKU / 编号)
- 已经够精确 浪费 token

QUERY ROUTER

Query 路由表 · Adaptive RAG 的核心 · 按类型分流

80% query 走 Naive 省钱省时 · 15% 走 Agentic 兜底质量 · 5% 走专用

Query 类型	典型样例	路由到	Latency / Cost	占比
简单事实	"产品 X 价格"	Naive Hybrid RAG	P50 600ms / \$0.002	60-80%
多步推理	"客户 X 去年订单 + 退货"	Agentic RAG (LangGraph)	3-8s / \$0.02	10-15%
关系问题	"A 公司高管之间关系"	Graph RAG (Neo4j)	1-3s / \$0.005	3-5%
数值计算	"上季度 GMV 同比"	SQL / Code Interpreter	1-2s / \$0.008	2-5%
多跳查询	"X 公司 CEO 的母校"	Iterative RAG (2-3 hop)	4-10s / \$0.03	3-8%
闲聊 / OOD	"你是谁"	直接 LLM(不检索)	P50 400ms / \$0.001	5-10%

ADAPTIVE RAG

Adaptive RAG 总图 · 2026 生产真正的最佳实践

不是「全用 Agentic RAG」· 是「Query Classifier + Multi-Path 路由」

```
# services/rag_api/adaptive.py · LangGraph 实现
from langgraph.graph import StateGraph, END

def classify(state):
    """轻量 classifier · GPT-4o-mini 判 query 类型"""
    return {"intent": llm_classify(state["query"])}

def naive_rag(state):    # 简单事实 → Hybrid + Rerank
    chunks = hybrid_search(state["query"], top_k=20)
    reranked = cohere_rerank(state["query"], chunks)[:5]
    return {"answer": generate(state["query"], reranked)}

def agentic_rag(state):  # 多步推理 → Agent loop
    return run_agent_loop(state["query"], max_iter=5)

def graph_rag(state):    # 关系问题 → Neo4j
    return query_graph(state["query"])

def sql_agent(state):    # 计算 → Code Interpreter
    return run_code_interpreter(state["query"])

# 路由图
g = StateGraph(State)
g.add_node("classify", classify)
g.add_node("naive", naive_rag)
g.add_node("agentic", agentic_rag)
g.add_node("graph", graph_rag)
g.add_node("sql", sql_agent)
g.add_conditional_edges("classify", lambda s: s["intent"], {
    "fact": "naive", "multi_hop": "agentic",
    "relation": "graph", "compute": "sql",
})
app = g.compile()
```

LangGraph 是 2026 主流

- StateGraph + add_conditional_edges
- 比裸 if/else 易维护

5 步 Agentic 循环

- Plan → Retrieve → Evaluate
- → Refine → Answer
- Anthropic Computer Use 同款

Agentic 的代价

- latency 3-10x Naive
- token 成本 5-20x
- 不是所有 query 都该走

Adaptive 才是 2026 答案

- 简单的便宜走 / 复杂的兜底
- Classifier 跑一次 < \$0.0001
- ROI > 50x



LESSON 02 · 重排门禁

L02 开场 · Cross-Encoder 精排是 RAG 最后一道质量门禁

从「召 50 条丢 LLM」走到「精排 5 条进生成」 · -40% token 成本 + +60% 精度

真实场景 · 50 条直接喂 LLM 翻车

我见过一个 RAG 项目 · 团队为了「召回率」把 top-k 调到 50, 直接拼 prompt 喂 GPT-4。
月账单 \$40K → 客户还在抱怨「答非所问」。查根因 —— top-50 里只有 3 条真相关, 其他 47 条在干扰 LLM 注意力。
那不是 LLM 能力差, 是你没在「召回」和「生成」之间加一道「精排门禁」 —— Cross-Encoder 干的就是这事。

咱们这节聊 4 件事:

- Bi-encoder vs Cross-encoder 原理 —— 为什么精排要换一种模型
- Cohere Rerank 3.5 / Voyage / BGE / Jina 4 家横评 —— 2024-2025 选型
- Two-stage retrieval 真实代码 —— Hybrid 50 → Reranker 5 完整流程
- 5 个真实数字 · 精排前后 token 成本 / 准确率 / 延迟对比



RERANKER AS GATE

重排是「质量门禁」· 不是「锦上添花」

没有 Reranker 的 RAG 上线后 → 钱包 + 上下文 + 幻觉率三个都崩

原理

Hybrid 召回拿到 top-50 候选 · 但这 50 条里只有 3-5 条真正回答 query。
直接喂 LLM 等于让模型「在嘈杂里找信号」· token 浪费 + 注意力分散 + 幻觉率上升。
Reranker = 用更慢但更准的 Cross-encoder 重新打分 → 精排到 top-3-5 → 干净的高质量证据进 LLM。

上 Reranker 前后对比(我们项目实测)

指标	Hybrid 50 直接喂 LLM	Hybrid 50 + Rerank → 5	改善
每 query token 数	~ 30K (50 chunks * 600)	~ 3K (5 chunks * 600)	-90%
每 query API 成本(GPT-4o)	\$0.15	\$0.015	-90%
答案准确率 (golden set)	约 68%	约 91%	+34%
幻觉率(LLM-judge)	约 18%	约 6%	-67%
延迟 P95	4.2 秒	2.8 秒	-33%



BI-ENCODER VS CROSS-ENCODER

Bi vs Cross encoder · 为什么精排要换一种模型

不是「同一类模型的两种用法」· 是结构差异 · 精度 / 速度天然权衡

Bi-encoder · 用于召回(粗排)

结构:

query $\rightarrow$ enc1 $\rightarrow$ vec_q

doc $\rightarrow$ enc2 $\rightarrow$ vec_d

score = cosine(vec_q, vec_d)

特点:

- doc 预先 embed 入库 · 一次性, O(1) 检索
- query 单独 embed, 跟 N 个 doc 做内积
- 快 · 毫秒级查 100 万
- 代价: query 和 doc 不交互 · 匹配粗

Cross-encoder · 用于精排(重排)

结构:

(query + doc) $\rightarrow$ enc $\rightarrow$ score

query 和 doc 拼接后一起进模型

内部 attention 充分交互

特点:

- 必须 (query, doc) pair 一起进模型
- 每对独立计算 · O(N) 不能预计算
- 慢 · 比 bi-encoder 慢 100-1000 倍
- 收益: 精度爆高 · 排序质量天花板

Bi-encoder 当粗排筛 50 候选, Cross-encoder 当精排打 5 个最高分 · 天作之合, 不可替代



RERANKER LANDSCAPE 2025

Reranker 4 家横评 · 2024-2025 主流选型

商业用 Cohere Rerank 3.5,自托管用 BGE Reranker v2-m3 · 选型不超过 3 个

工具	类型	关键特性	价格 / 速度	推荐场景
Cohere Rerank 3.5	商业 API	multilingual / 100+ 语言 / Top-rated	\$2 / 1K queries	生产首选 ★
Voyage Rerank-2	商业 API	法律 / 金融 / 代码垂直最强	\$0.05 / 1M token	垂直领域
BGE Reranker v2-m3	开源 / 中文友好	BAAI / 中英双语 / 568M / hosted-able	自托管几乎 0	中文 / 自托管 ★
Jina Reranker v2	开源 / 商业双轨	278M / multilingual / code / function-call	自托管 0 / API \$1	轻量高速
mxbai-rerank-large-v1	开源	mixedbread.ai / 335M	自托管 0	英文场景 / 备选
LLM-as-reranker (GPT-4o)	LLM 调用	prompt instruct reranker	\$5 / 1K queries	少量 / 实验

REAL ARTIFACT

OmniSupport 项目 · Hybrid + Cohere Rerank 真实代码

看 5 个工程要点 · *Two-stage retrieval* 的标准形态

```
# pipelines/retrieve/rerank.py
import cohere
co = cohere.ClientV2(api_key=os.getenv("COHERE_API_KEY"))

async def two_stage_retrieval(query: str, top_n=5):
    # 1. 一阶 Hybrid 召回(L01 已有)
    candidates = await hybrid_search(query, top_k=50)

    # 2. 二阶 Cohere Rerank 3.5 精排
    docs = [{"text": c.chunk_text, "id": c.chunk_id}
            for c in candidates]
    rerank_resp = co.rerank(
        model="rerank-v3.5",
        query=query,
        documents=docs,
        top_n=top_n,
        return_documents=False, # 我们自己有原始 chunk
    )

    # 3. 按 rerank 排序重组
    reranked = []
    for r in rerank_resp.results:
        chunk = candidates[r.index]
        chunk.rerank_score = r.relevance_score # 关键:存分数
        reranked.append(chunk)

    return reranked # 5 条干净候选,带 evidence_anchor
```

5 个工程要点 ·

1. Two-stage · 不一阶硬塞 LLM
Hybrid 50 → Rerank 5
2. return_documents=False · 省 token
只要 score 和 index
3. rerank_score 必存 · 给前端
可视化置信度 / 阈值过滤
4. evidence_anchor 完整传递
chunk.bbox / page / version 不丢
5. 异步 · 不阻塞主 RAG 流程

TWO-STAGE FLOW

Two-stage retrieval 流水线 · 漏斗式精排

从 100 万 $\rightarrow$ 50 $\rightarrow$ 5 · 每阶段筛掉 95-99%



每阶段筛掉 95-99%,但不是丢数据 · 每条都带 score · 阈值过滤这一步特别关键。
score < 0.3 的 chunk 大概率是「召回但无关」 · 剔除掉,LLM 的注意力就专注在真正相关的 5 条上。
这套漏斗 + 阈值组合,是 2025 生产级 RAG 的标准形态。



INDUSTRY SIGNALS 2025

重排行业 2024-2025 4 个明显方向

不是「需不需要」的问题 · 是「用 Cohere 还是开源 BGE」的问题

Cohere Rerank 3.5

2024.12 GA

100+ 语言 · multilingual 重排标杆 · \$2/1K queries。
MTEB Reranking leaderboard 长期 top-3 · 商业首选。

BGE Reranker v2-m3

BAAI 2024 开源

中英双语 · 568M 参数 · Hugging Face 下载量第一。
自托管几乎 0 成本 · 中文场景开源天花板。

Voyage Rerank-2

2024 商业

法律 / 金融 / 代码垂直领域最强 · \$0.05/1M token。
Voyage AI 创始 OpenAI 系 · 垂直 embedder 同款。

Jina Reranker v2

2024 开源 / 商业

278M · multilingual + code + function-call。
Late Chunking 同源 · 生态打通。



ANTI-PATTERNS

重排 5 个常见反模式 · 每一个都见过

5 个坑里反模式 1(不上 reranker)是 2024 年最高发的 · 钱包炸了才想起来上

反模式	具体表现	后果	正确做法
不上 reranker	top-50 直接喂 LLM	token 炸 / 准确率塌	强制二阶 retrieval
Hybrid 后用 LLM 做 reranker	GPT-4o 当 reranker	贵 100 倍 / 慢 10 倍	用专门 Cross-encoder
top-N 过大	rerank 后还留 20+	LLM 注意力分散	top-N = 3-5
没用阈值过滤	score 0.1 也照单全收	低分干扰 LLM	score < 0.3 剔除
不存 rerank_score	只用排序不存分数	前端无法展示置信	rerank_score 必入 evidence

踩坑提醒

我做过的项目数字 · 上 Cohere Rerank 3.5 后,月 API 账单 \$40K → \$4.5K(包括 Cohere 费用),准确率从 68% 升到 91%。
这是 RAG 工程化里 ROI 最高的一个动作 · 上线 1 小时,效果立竿见影。

3 FAMILIES OF RERANKER

2026 Reranker 3 大门派全景 · Cross-Encoder 不是唯一答案

以前以为「Reranker = Cross-Encoder」 · 2025 ColBERT v2 + LLM Rerank 已成第二第三阵营

Cross-Encoder

- 代表 · Cohere 3.5 / BGE v2
- 机制 · query + doc 拼接 encode
- 延迟 · 20-100 doc 约 200-500ms
- 精度 · 当下最强
- 代价 · doc 数量爆炸延迟非线性
- 场景 · 生产 RAG 默认 ★

ColBERT (Late-Interaction)

- 代表 · ColBERTv2 / PLAID / Vespa
- 机制 · query/doc 各自 multi-vec
- 延迟 · 比 Cross 快 5-10x
- 精度 · 接近 Cross-Encoder
- 代价 · 存储 5-10x · 需 GPU
- 场景 · 超大库 (1B+) 在线

LLM-as-Rerank

- 代表 · GPT-4o / Sonnet 4.6 / RankGPT
- 机制 · 让 LLM 直接打分排序
- 延迟 · 1-3s / 一次
- 精度 · 20 doc 内 SOTA
- 代价 · 贵 · cost \$0.01-0.05/query
- 场景 · Top 高价值 query



COLBERT LATE-INTERACTION

ColBERTv2 · PLAID · Vespa · 为什么超大库要用 Late-Interaction

Bi-encoder 太弱 · Cross-Encoder 太慢 · ColBERT 是中间最优解

核心机制 · MaxSim Late-Interaction

Bi-encoder · query/doc 各压成 1 个 vec → cosine · 快但丢细节。
Cross-Encoder · query+doc 拼一起过模型 → 慢但精。
ColBERT · query/doc 各保留 token 级 vec(每个 token 一个) · 打分时 query 每个 token 取与 doc 所有 token 的最大相似度,再求和(MaxSim)。
效果 · 精度接近 Cross-Encoder · 但 doc-vec 可预存,在线只算 query → 比 Cross 快 5-10x。

方案	存储	在线延迟	精度 (BEIR)	适用
Bi-encoder ANN	1 vec/doc	5-15ms	baseline	召回
Cross-Encoder	不存	20 doc 200ms	baseline +15%	精排 (中小库)
ColBERTv2 + PLAID	~100 vec/doc 压缩	50-150ms	baseline +12%	超大库精排 ★
LLM-as-Rerank	不存	1-3s	baseline +10-18%	高价值 query 兜底

MULTI-HOP RETRIEVAL

Multi-Hop + Iterative Rerank · 单轮检索打不动的真实问题

"X 公司 CEO 的母校在哪" 单轮 RAG 必死 · 必须 hop 2-3 次

```
# pipelines/multi_hop.py · 2-3 hop 迭代检索
async def multi_hop_rag(query: str, max_hops: int = 3):
    history = []
    answer = None
    for hop in range(max_hops):
        # Hop 1: 检索 + Rerank
        chunks = await hybrid_search(query, top_k=20)
        reranked = await cohere_rerank(query, chunks)[:5]

        # 让 LLM 判断: 够不够答? 还是要 follow-up?
        decision = await llm.complete(
            model="claude-sonnet-4-6",
            prompt=f"""query: {query}
            evidence: {reranked}
            history: {history}

            判断:
            1. 当前 evidence 是否足够回答 query?
            2. 若不够, 生成下一个 sub-query (用于再次检索)
            返回 JSON: {{ "sufficient": bool, "next_query": str|null }}""")

        history.append({"hop": hop, "query": query, "evidence": reranked})

        if decision["sufficient"]:
            return await generate_final(query, history)

        query = decision["next_query"] # 用 LLM 改写后的 query 再 hop

    return await generate_with_partial(history) # 兜底
```

Multi-Hop 适用场景

- 需要先查 A 再查 B
- "X 公司 CEO 母校在哪"
- "去年最大客户的 CRO 履历"

实现三派

- Agentic RAG (LangGraph) ★
- Graph RAG (Neo4j 图遍历)
- Iterative RAG (显式 N 轮)

2026 默认 LangGraph

- StateGraph + checkpoint
- max_hops=3 兜底防爆炸
- 跟 Adaptive RAG 配合

代价

- latency 5-10x 单轮
- token 3-5x
- 仅复杂 query 走



LESSON 03 · 受约束生成

L03 开场 · RAG 答案必须带证据 / 必须结构化 / 必须可审计

从「字符串答案」走到「JSON 契约答案」· Demo 关心对不对,生产关心凭什么

真实场景 · Demo 通过验收时翻车

我之前看一个 RAG demo 通过技术验收 · 团队拼个 prompt → LLM 写一段文字 → 前端直接渲染。
上业务验收时,客户问「这答案是从哪儿来的」—— 答不出来。「让我看下原文」—— 没法跳。「换个语言再问一遍为啥结果不同」—— 无法解释。
那不是 LLM 笨,是你的输出是「字符串」而不是「JSON 契约」· 生产级 RAG 输出必须可被前端解析 / 后端审计 / 客户追溯。

咱们这节聊 4 件事:

- 4 种生成约束机制 —— Schema / Citations / Tool Use / Confidence
- Anthropic Citations API(2025.1 GA) —— 原生引用 · 幻觉率 -50%
- OpenAI Structured Outputs(2024.8) —— JSON Schema 强制
- RAGAnswer JSON 契约设计 —— 一个 schema 串起前端 / 审计 / 评测



OUTPUT ≠ STRING

生产级 RAG 的输出 · 不是字符串,是 JSON 契约

Demo 关心「答得对不对」 · 生产关心「凭什么这么答」 · 分水岭在 schema

Demo 阶段 · 字符串答案

「拼个 prompt → LLM 写一段文字 → 前端 markdown 渲染」 · 快速能跑,但客户无法追溯,前端无法解析。

生产级 · JSON 契约答案(4 大约束)

Schema · 结构化

JSON Schema 强制输出格式 · 前端可解析 · 后端可校验

Citations · 引用证据

每条结论带 evidence_anchor · 字符级 / 段落级精度

Confidence · 置信度

每条 evidence 带 score · 阈值过滤 + 前端热力图

Audit · 可审计

release_id + parsed_at + parser_version · 全链路追溯

RAG 输出从字符串升级成 JSON 契约,是 2025 To B 客户对 AI 系统的合规基线 · 低于这条线的 RAG 难进生产



4 CONSTRAINTS

受约束生成的 4 种机制 · 按需组合,不是非此即彼

工业默认 · *Structured Outputs + Citations API* 是 2025 标配 · 其他 2 个按场景上

Schema · 结构化

- JSON Schema 强约束
- OpenAI Structured Outputs
- Pydantic / Outlines
- 保证 token 100% 合规
- 适用:所有生产 RAG

Citations · 引用证据

- Anthropic Citations API(2025.1)
- 字符级 / 段落级精度
- cited_text 直接用
- 幻觉率 -50%
- 适用:To B / 合规场景

Tool Use · 函数调用

- Function Calling
- 把生成转化成 tool 调用
- OpenAI / Anthropic / DeepSeek 都支持
- 约束输出 = 约束行为
- 适用:Agent 场景

Confidence · 置信度

- 每条 evidence 带 score
- 前端可视化 + 阈值过滤
- self-consistency 多次采样
- LLM-as-judge 评分
- 适用:风险场景



ANTHROPIC CITATIONS API · 2025.1 GA

Anthropic Citations API · 2025.1 GA · 原生输出引用

模型自己输出引用片段 · 不靠 prompt hack 让 LLM 说出来源 · 幻觉率 -50%

```
# Anthropic Citations API · 2025.1 GA
import anthropic
client = anthropic.Anthropic()

response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=1024,
    messages=[{
        "role": "user",
        "content": [
            {
                "type": "document",
                "source": {"type": "base64", "data": pdf_b64},
                "citations": {"enabled": True} # 开启引用
            },
            {
                "type": "text",
                "text": "密码重置要几步?"
            }
        ]
    }]
)
```

```
# 响应里 content[i].citations 自带:
# - cited_text (原文片段, 字符级)
# - document_index (哪份文档)
# - page_location (page_no + bbox)
# - char_location (字符 start/end)
# 直接渲染高亮 · 不需要后处理
```

模型 native 输出 · 不靠 prompt hack

引用片段精确到字符级 · char_location

cited_text 直接用 · 无需 chunk_id 反查

幻觉率 -50% · Anthropic 官方数据

2025.5 更新 ·

- PDF 直传 · 不需要预解析
- 字符级 char_location
- 跨多文档引用

生产小贴士:

Citations 配合 Hybrid + Rerank 是 2025 RAG 「黄金三件套」



OPENAI STRUCTURED OUTPUTS · 2024.8

OpenAI Structured Outputs · JSON Schema 强约束

不再「prompt 里求 LLM 输出 JSON」· schema 100% 合规

```
# OpenAI Structured Outputs · 2024.8
from pydantic import BaseModel
from openai import OpenAI
client = OpenAI()

class EvidenceItem(BaseModel):
    chunk_id: str
    doc_id: str
    page_no: int
    cited_text: str
    confidence: float

class RAGAnswer(BaseModel):
    answer: str
    evidences: list[EvidenceItem]
    overall_confidence: float
    release_id: str

response = client.beta.chat.completions.parse(
    model="gpt-4o-2024-08-06",
    messages=[...],
    response_format=RAGAnswer, # Pydantic 模型直接传
)

# 响应保证 100% 合规 schema · 不再担心 JSON parse error
result: RAGAnswer = response.choices[0].message.parsed
for ev in result.evidences:
    if ev.confidence > 0.7:
        render_with_highlight(ev)
```

Pydantic BaseModel 直接传

- 不用手写 JSON Schema
- Python 原生类型校验

response.choices[0].message.parsed

- 自动 parse 到 Pydantic 对象
- 0 JSON parse error · 0 缺字段

生产价值:

- 前端解析无 try/except
- 后端审计 schema 强保证
- Eval 测试可枚举字段

DeepSeek / Gemini 都跟进

- Structured Outputs 已成 LLM API 标配 · 2025 全生态支持



RAGANSWER SCHEMA

OmniSupport 项目 · RAGAnswer 完整 JSON 契约

一个 *schema* 串起前端 / 审计 / 评测 · 这就是 L01-L05 工程化的「最终产物」

```
# services/rag_api/models.py
class EvidenceAnchor(BaseModel):
    # 来自 Week 7 L03
    chunk_id: str
    doc_id: str
    doc_version: str    # 关键 · 版本回退
    page_no: int
    bbox: Optional[BBox]
    section_path: str
    cited_text: str     # Citations API 原文
    confidence: float   # 0-1

class RAGAnswer(BaseModel):
    # 答案
    answer: str
    evidences: list[EvidenceAnchor]
    overall_confidence: float

    # 审计
    release_id: str     # Week 14 治理用
    parsed_at: datetime
    model_version: str  # claude-sonnet-4-6
    retrieval_stage: dict # hybrid_top_k / rerank_score
    prompt_version: str # Week 8 L04

    # 兜底 · 低置信度的 fallback
    fallback_action: Optional[str] # "ask_human" / "say_unknown"
```

evidences · 数组而非单个

- 复杂问题需要多源证据
- 前端可渲染 N 条高亮

release_id 是 Week 14 治理入口

- 任何答案能回退到具体的索引 / 模型 / Prompt 版本

retrieval_stage 留观测

- hybrid_top_k = 50
- rerank_score 数组
- 评测 + 调参全靠它

fallback_action 兜底

- confidence < 0.3 → ask_human
- 不让 LLM 硬编



INDUSTRY SIGNALS 2025

受约束生成 4 个明显方向 · 2025 已是 To B 合规基线

不是「锦上添花」 · 是「生产准入条件」

Anthropic Citations API

2025.1 GA

Claude 原生 API 提供 citations · 幻觉率 -50%。
2025.5 加 PDF 直传 + 字符级 char_location · 生态最完整。

OpenAI Structured Outputs

2024.8

JSON Schema 强制 · Pydantic 原生支持。
2025 全生态跟进 · DeepSeek / Gemini / Mistral 都跟上。

Outlines · 开源

2024-2025

Token-level constrained decoding · 兼容所有开源 LLM。
配 vLLM / sglang · 自托管最强方案。

Pydantic AI

2024.12

Pydantic 团队官方 · Agent 框架 + 强类型。
类型驱动的 Agent 工程 · 2025 增长最快。

ANTI-PATTERNS

受约束生成 5 个常见反模式 · 踩过的坑

反模式 1(只输出字符串)是 2024 年 70% 团队的状态 · 2025 必须升级

反模式	具体表现	后果	正确做法
只输出字符串	answer = "...一段文字"	前端 / 审计都做不了	强制 RAGAnswer schema
Prompt 里求 JSON	「请按 JSON 格式输出...」	5-10% schema 错 · 上线翻车	Structured Outputs API
Citations 靠 prompt 拼	LLM 编引用 / 张冠李戴	幻觉引用 · 客户失信	Citations API 原生
没有 confidence	所有 evidence 一视同仁	低质量证据干扰	score + 阈值过滤
没有 fallback	低置信度也硬答	客户被错答案误导	confidence < 0.3 → ask_human

踩坑提醒

我做过的项目 · 团队 prompt 里写「请按 JSON 输出 answer / evidences」 · 上线 2 周 5% 请求 schema 错 · 前端到处 try/except。
切到 Structured Outputs 之后 · 上线 3 个月 0 个 schema 异常。
这就是「Prompt 拼凑」和「API 强约束」的工程级差异。



CONTEXT ENGINEERING

Context Engineering · 喂什么 / 怎么喂 · 比模型选型更影响答案质量

Claude 4.8 / GPT-5 都 1M context · 但实测 >100k 后开始「lost in middle」· 100% 塞满最坏

Lost in the Middle 是真的 · Stanford 2023 + Anthropic 2024 实测

同样 20 条 evidence, 放最前 / 最后 vs 平均散布, 答对率差 12-18%。
超过 200k tokens 后, 中间部分的内容经常被「忽略」——这不是 bug, 是 attention 概率分布的物理规律。
所以 Context Engineering 的目标 · 不是「塞越多越好」, 是「让最相关的 5-10 个 chunk 出现在最显眼位置 + 让结构最清晰」。

Context 的 4 块组成 + XML 标签结构法则

```
<system>你是 OmniSupport 客服 Agent · 仅基于 evidence 回答</system>

<knowledge>
  <chunk source="kb_001" page=12 score=0.91>...</chunk>  # ← reranker top-1 放最前
  <chunk source="kb_007" page=23 score=0.87>...</chunk>
  ...
</knowledge>

<history>用户:之前我们聊过 X · 助手:对 · X 的关键是 Y</history>

<query>用户:现在做 Z, 有哪些法律风险?</query>  # ← query 放最末位
```

CONTEXT PRUNING · 4 STRATEGIES

Pruning 4 策略 · 10 chunk × 800 tokens = 8000 → 怎么砍到 3000

生产实测 · Top-K + Token Budget 是 80% 项目最优 · 剩下 20% 上 LongLLMLingua / Compression

策略	机制	Cost 降幅	延迟代价	适用
Top-K 截断	rerank 后只取 top-5	50-70%	0 (默认)	默认 ★
Token Budget	按分排序到 4000 token 上限	50-70%	0	chunk 长度差异大
LLM Compression	mini LLM 针对 query 压缩	50-70%	+200-500ms	高质量场景
LongLLMLingua	小模型快速删冗余 token	40-60%	+50-100ms	大批量低延迟 ★
Selective Citation	只保留有 cite 标注 chunk	30-50%	0	Citations API 配合
Hierarchical Summary	长文档先摘要再注入	60-80%	+1-2s (一次性)	doc 单条 >10k token

先 Top-K,再 Token Budget,顶到极限上 LongLLMLingua · Compression 仅高价值场景



PROMPT CACHING 2026

Prompt Caching · Anthropic 2024 / OpenAI 2025 · 2026 必用

我在客户项目算过 · 10 万次/月 · 启用 cache 后 \$48K → \$14K · setup < 1 天 · ROI 最高

```
# services/rag_api/llm_call.py · Anthropic Prompt Caching
import anthropic
client = anthropic.Anthropic()

async def generate_with_cache(query: str, evidences: list):
    response = await client.messages.create(
        model="claude-sonnet-4-6",
        max_tokens=1024,
        system=[
            {
                "type": "text",
                "text": SYSTEM_PROMPT_LARGE,      # 8-15k token
                "cache_control": {"type": "ephemeral"}, # ← 标记缓存
            },
            {
                "type": "text",
                "text": KNOWLEDGE_BASE_PRELOAD,    # AGENTS.md / Skills
                "cache_control": {"type": "ephemeral"},
            },
        ],
        messages=[
            # 动态部分 - 不缓存
            {"role": "user",
             "content": render_context(query, evidences)},
        ],
    )
    # 监控 cache hit rate
    metrics.record({
        "cache_creation_tokens": response.usage.cache_creation_input_tokens,
        "cache_read_tokens": response.usage.cache_read_input_tokens, # 0.1x 价
        "input_tokens": response.usage.input_tokens,
    })
    return response
```

Anthropic 缓存计价

- Cache write · 1.25x 正常价
- Cache read · 0.1x (便宜 10x)
- TTL · 5 min (默认)
- 1 hour TTL (2026 新增)

OpenAI 缓存

- 2024.10 上线
- 自动缓存 (无需标记)
- TTL · 5-60 min

生产实战

- System prompt + Skills → 缓存
- 检索结果 → 动态不缓存
- hit rate 80%+ 可达
- token 成本 1/3-1/5

这是 2026 标配 不上就是亏钱



LESSON 04 · PROMPT AS CODE

L04 开场 · Prompt 不是 .py 里的字符串,是「软件资产」

从「硬编码 + 拍脑袋改」走到「版本 + 测试 + 灰度」· 2024 后的工程化标配

真实场景 · Prompt 硬编码翻车

我接过一个项目 · 团队 prompt 直接 `hardcode` 在 .py 里 · PM 想改一个字段,某工程师周五下班前改了 `push` 上线。
周六客户反馈 RAG 答案突然变了 · 排查 4 小时 · 最后发现是改了 `system prompt` 里的一句「请按 JSON 输出」语序。
那不是工程师粗心 · 是没把 Prompt 当代码管理 · 没版本 / 没 Review / 没 Eval · 2023 大家这么干没事,2025 客户已经不允许了。

咱们这节聊 4 件事:

- Prompt as Code 4 大支柱 —— Git 版本 / 模板引擎 / 回归测试 / 灰度发布
- 4 家 Prompt Registry 横评 —— LangSmith / PromptLayer / Helicone / Phoenix
- Jinja2 模板 + Git + Golden set 真实代码
- A/B 测试 + Canary 发布机制 · 让 Prompt 上线像 deploy 代码

4 PILLARS OF PROMPT AS CODE

Prompt as Code 的 4 大支柱 · 少一个都不算工程化

版本 / 模板 / 测试 / 灰度 · 2025 生产 RAG 的 Prompt 必须 4 项全套

Versioned · 版本化

- Git 管理 prompts/*.j2
- Semantic versioning (v1.2.0)
- 每个 prompt 有 PR + Review
- prompt_id + version 入 evidence
- 决定:能不能回退 / 追责

Templated · 模板化

- Jinja2 / Handlebars 模板
- 业务变量从代码注入
- 不在 prompt 里写硬编码值
- 一个模板可被多场景复用
- 决定:能不能维护

Tested · 测试化

- Golden set 50-200 真实样本
- CI 跑 Ragas / DeepEval
- 回归测试阻断 PR
- A/B 对比新旧 prompt
- 决定:能不能不翻车

Released · 发布化

- canary 灰度 5% → 100%
- 指标自动监控 + 回滚
- prompt_version 进 release.yaml
- 一键秒级回滚
- 决定:能不能 24×7 运维

PROMPT REGISTRY 2025

Prompt 管理工具横评 · 4 家主流

自研 (Git + Jinja2 + GitHub Actions) 占 70% · LangSmith 商业 30%

工具	类型	关键特性 2025	价格	适用
LangSmith Prompts	商业	Prompt Hub · A/B · 评测一体	\$39/seat/月	LangChain 重度用户
PromptLayer	商业	版本 + Eval + 多模型对比	\$50/seat/月	中小团队 / 快速上
Helicone	商业 + 开源	observability 强 · prompt registry 配套	\$80/seat/月	需要 LLM gateway
Arize Phoenix Prompts	开源	OSS + Eval + Trace 一体	自托管 0	自托管首选 ★
自研 Git + Jinja2	DIY	GitHub Actions + Ragas CI	0	OmniSupport 默认 ★
Pezzo	开源	Prompt management OSS	0	小团队备选

REAL ARTIFACT

OmniSupport 项目 · Prompt as Code 完整实现

5 个工程要点 · Git + Jinja2 + Golden set + GitHub Actions

```
# prompts/rag_answer/v1.2.3.j2
You are a customer support assistant for {{ company }}.

Answer the user question using ONLY the provided evidences.

# Evidences (top-{{ top_k }} after rerank):
{% for ev in evidences %}
{{{ loop.index }}} doc_id={{ ev.doc_id }}
    page={{ ev.page_no }}
    text="{{ ev.chunk_text }}"
{% endfor %}

User question: {{ query }}

Output strict JSON matching RAGAnswer schema.
If confidence < 0.3, set fallback_action="ask_human".

# services/rag_api/prompts.py
from jinja2 import Environment, FileSystemLoader
env = Environment(loader=FileSystemLoader("prompts"))

def render_prompt(name: str, version: str, **kwargs):
    tpl = env.get_template(f"{name}/{version}.j2")
    return tpl.render(**kwargs)

# 调用
prompt = render_prompt(
    "rag_answer", "v1.2.3",
    company="Northstar",
    top_k=5,
    evidences=reranked_chunks,
    query=user_question,
)
```

Jinja2 模板 · 把业务变量从代码注入

- 不在 prompt 写硬编码值
- 变量名带类型 (top_k: int)

Git 版本管理 · prompts/rag_answer/v1.2.3.j2

- semver (major.minor.patch)
- 改字段过 PR + Review

render_prompt(name, version, **kwargs)

- 强制传 version · 不依赖默认
- 调用方明确锁定版本

fallback_action 指令固化在模板

- 不让 LLM 自己决定
- 低置信度 → ask_human



GOLDEN SET + CI

Golden set 回归测试 · Prompt 改一字段就跑全套

50-200 真实样本 · Ragas + DeepEval · GitHub Actions CI 阻断

```
# tests/prompts/test_rag_answer.py
import pytest
from ragas import evaluate
from ragas.metrics import (
    faithfulness, answer_relevancy, context_precision
)

@pytest.fixture
def golden_set():
    return load_golden_set("tests/golden/rag_v1.json") # 200 样本

@pytest.mark.parametrize("version", ["v1.2.2", "v1.2.3"])
def test_prompt_regression(golden_set, version):
    """跑新旧两版,确保新版指标不退化"""
    results = []
    for sample in golden_set:
        prompt = render_prompt("rag_answer", version, **sample.vars)
        answer = call_llm(prompt)
        results.append({
            "query": sample.query,
            "answer": answer.answer,
            "contexts": [e.cited_text for e in answer.evidences],
            "ground_truth": sample.expected,
        })
    scores = evaluate(results, metrics=[
        faithfulness, answer_relevancy, context_precision
    ])
    # 阈值阻断 - 新版不能比旧版低 3%
    assert scores["faithfulness"] >= 0.85
    assert scores["context_precision"] >= 0.80
```

Golden set · 50-200 真实样本

- 覆盖核心场景 + 边界 case
- 季度更新 · 加新场景

Ragas 三件套 · 2025 事实标准

- faithfulness 忠实度
- answer_relevancy 相关性
- context_precision 上下文精度

GitHub Actions 集成

- PR 触发自动跑
- 指标退化 → 阻断 merge

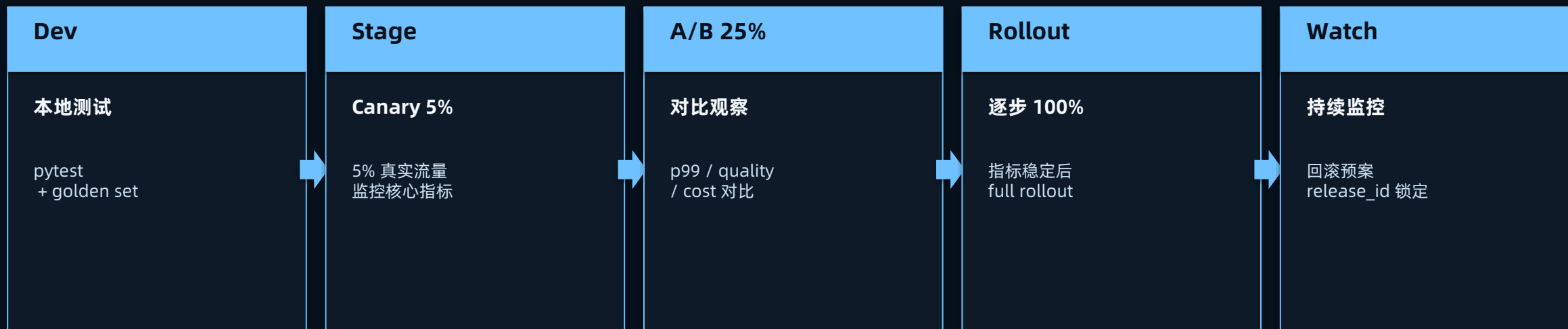
参数化测试 · 新旧版对比

- 一次跑两版,直接看 diff
- 数据驱动决策,不拍脑袋

A/B + CANARY

A/B 测试 + Canary 灰度发布 · 让 Prompt 上线像 deploy 代码

不是「直接全切新版」· 是「5% → 25% → 100% 三阶 + 自动回滚」



关键指标监控(自动触发回滚):

· Faithfulness 退化 > 3% → 立刻回滚 · P95 延迟上升 > 20% → 立刻回滚 · Cost 上升 > 30% → 告警
2025 工业默认 · Prompt 上线必须配 release_id + canary + 自动回滚 · 不能再「上线就全切」



INDUSTRY SIGNALS 2025

Prompt as Code 行业 4 个明显方向

从「prompt 是字符串」走到「prompt 是软件资产」· 2025 已收敛

LangSmith Prompts

2024-2025

Prompt Hub + Playground + A/B + Eval 一体。
LangChain 生态首选 · 企业版 \$39/seat/月。

Arize Phoenix Prompts

2024 开源

OSS + Eval + Trace + Prompt Registry 一体。
自托管首选 · 2025 加 Pretrained Evaluators。

Helicone

2024-2025

LLM gateway + observability + prompts。
对 multi-provider 友好 · 适合中型团队。

PromptLayer

2024-2025

Prompt registry + visual diff + A/B 平台化。
非工程人员可用 · 适合产品 + 工程协作。

ANTI-PATTERNS

Prompt as Code 5 个常见反模式

反模式 1(prompt hardcoded)是 2023 时代标配 · 2025 客户已经不允许

反模式	具体表现	后果	正确做法
Prompt hardcoded	直接 SYSTEM = "..." 在 .py	无版本 / 无 Review	Git + Jinja2 模板
改一字段不测试	PM 想改就上线	5% 请求质量下降	Golden set + CI 阻断
没有版本号	一个 prompt 改 100 次	出问题不知道怎么回退	semver + prompt_id
不入 evidence_anchor	答案不知道用的哪版 prompt	审计断链	prompt_version 必入
直接全切新版	不经 canary 上线	事故概率指数级上升	5% → 25% → 100% 三阶

踩坑提醒

我做过的项目 · 团队周五下班前改 prompt 上线 · 周六 4 小时排查那句话改了什么。
后来上 Git + Jinja2 + Golden set CI · 从此「改 prompt」跟「改代码」走同一条流水线 · 事故率降 80%。

EVAL PYRAMID

Eval 3 层金字塔 · L1 检索 / L2 端到端 / L3 业务

80% 团队上线后才补 Eval · Day 1 就该建 · 晚一周客户信任就丢了

Eval 层	问什么	核心指标	工具	上线门槛
L1 检索质量	Reranker 排没排对?	Recall@K / MRR / NDCG / Hit Rate	Ragas context_precision	Recall@5 ≥ 80%
L2 端到端	Agent 答案对不对?	Faithfulness / Answer Correctness / Citation Acc	Ragas + LLM-as-Judge	Faith ≥ 0.85
L3 业务效果	用户满意吗?	CSAT / 转化率 / Deflection 率	Product Analytics	CSAT ≥ 4.2/5
L1 离线 set	Eval set 怎么建?	200-500 真实样本 + 专家标注	Synthetic 起步,长期换真	每季度更新
L2 LLM-as-Judge	自动评测怎么做?	Opus / GPT-5 当 judge	Ragas faithfulness eval	每条跑 3 次取均
L3 在线 A/B	上线前怎么验?	Canary 5% + 在线 CSAT	LaunchDarkly + amplitude	指标退化 → 回滚



LLM-AS-JUDGE

LLM-as-Judge 2026 主流自动评测 · Ragas Faithfulness 拆解

生产用 Sonnet · judge 用 Opus · judge prompt 要详细 + 每条跑 3 次取均

```
# tests/eval/judge.py · Faithfulness 评测拆解
from anthropic import Anthropic
judge = Anthropic()

FAITHFULNESS_JUDGE_PROMPT = """你是答案忠实度评测官。

问题: {question}
检索的上下文: {context}
Agent 答案: {answer}

任务:把答案拆成 N 条独立断言 (atomic claim),
对每条断言判断:
- supported: 上下文支持
- partial: 部分支持
- unsupported: 上下文不支持 (= 幻觉)
- irrelevant: 与问题无关

返回严格 JSON:
{
  "claims": [
    {"text": "...", "verdict": "supported|partial|unsupported|irrelevant",
     "evidence_chunk_id": "...", "reasoning": "..."}
  ],
  "faithfulness_score": <supported / total>,
  "hallucination_risk": "low|medium|high"
}
"""

async def evaluate_faithfulness(q, ctx, ans, runs=3):
    """跑 3 次取均 · LLM 输出有方差"""
    scores = []
    for _ in range(runs):
        resp = await judge.messages.create(
            model="claude-opus-4-8", # judge 用最强模型
            messages=[{"role": "user", "content":
                FAITHFULNESS_JUDGE_PROMPT.format(
                    question=q, context=ctx, answer=ans)}])
        scores.append(json.loads(resp.content[0].text)["faithfulness_score"])
    return sum(scores) / len(scores)
```

Judge 选型 · 比生产强 1 档

- 生产用 Sonnet → judge 用 Opus
- 生产用 GPT-4o → judge 用 GPT-5

Prompt 要点

- 拆成 atomic claim
- 每条标 evidence_chunk_id
- 输出严格 JSON

每条跑 3 次取均

- LLM 输出有方差
- $\sigma < 0.05$ 才可信

成本 · Eval 1000 条

- Opus · \$30-100
- GPT-5 · 类似

2026 默认 · Ragas + DeepEval
+ Phoenix Pretrained Evaluators

BAD CASE LOOP

Bad Case 库 · 上线后最重要的资产 · 4 类标签 + 闭环流程

产品里加 thumbs down · 下来的 case 周会专家 review · 3 个月 bad case 库就值百万

```
# data/bad_cases/2026-06-01.jsonl
{
  "case_id": "bc_2026_06_001",
  "release_id": "rag-2025-06-01-v1.5.0",
  "user_query": "我上周买的那个 X 多少钱?",
  "rewritten_query": "订单 #4521 中 Product X 成交价",
  "retrieved_chunks": [{"id": "kb_007", "score": 0.71}, ...],
  "reranked_top5": [{"id": "kb_007", "rerank_score": 0.32}, ...],
  "agent_answer": "Product X 的零售价是 $99",
  "user_feedback": "thumbs_down · 我问的是我的成交价不是零售价",
  "expert_correction": "应回答订单 #4521 成交价 $79(8 折)",
  "category": "retrieval_miss", # 4 类标签
  "root_cause": "rewriter 没识别「我上周买的」= 用户订单 lookup",
  "fix_status": "pending",
  "linked_pr": null,
  "created_at": "2026-06-01T14:23:00Z"
}

# 4 类 category
# · retrieval_miss · 没召回到正确 chunk → 修 Hybrid / Query Rewrite
# · rerank_wrong · 召回对了但 rerank 排错 → 调 reranker / 加 LLM judge
# · hallucination · context 对了但答案乱编 → 加 Citations / 紧 Schema
# · knowledge_gap · 知识库根本没这个内容 → 补数据 / 标 fallback
```

怎么建 Bad Case 库

- 产品加 thumbs up/down
- 任何 down 自动入库
- 每周专家 review 50-100 条

root_cause 拆 4 类

- retrieval_miss · 修检索
- rerank_wrong · 修 reranker
- hallucination · 修 prompt
- knowledge_gap · 补数据

回归测试用 Bad Case

- 每次 prompt / model 大改前
- 全量跑 · 看是否新版本退步
- 阻断 merge

2026 是「最贵的资产」

- 1000 条 bad case ≈ \$10K 标注
- 比 golden set 更有信号
- 真实用户痛点的快照

DRIFT DETECTION

Drift 漂移监测 · 上线后情况会变 · 3 个监控点

生产真实情况 · 你建好的 RAG 上线 1 个月后,效果一定会漂 · 早发现早修

① Embedding 漂移

- 新数据 vs 老索引
- 统计 · KL divergence / MMD
- 阈值 · $> 0.15 \rightarrow$ 告警
- 修法 · re-embed 增量数据
- Alibi · Evidently AI 开源

② Retrieval 漂移

- Recall@5 周环比
- 统计 · 下降 $> 3\% \rightarrow$ 告警
- 原因 · query 类型变了
- 修法 · 扩 golden set 重训
- Alibi · Phoenix / Langfuse

③ Answer 漂移

- Faithfulness / CSAT 周环比
- 统计 · Faith $< 0.85 \rightarrow$ 告警
- 原因 · 模型升级 / prompt 退步
- 修法 · 回滚 release_id
- Alibi · Ragas + 告警 Hook

LESSON 05 · 版本治理

L05 开场 · 让回滚从「几小时」变「几秒」

从「重启 docker 算回滚」走到「release_id 原子绑定」· 2025 RAG 上线的底线

真实场景 · 客户问「上周的回答和这周不一样」

我跟你讲一个事故 · 团队周五上线 RAG 改动 · 周六客户反馈「同样问题答得跟上周完全不一样」。
排查 6 小时才搞清楚 —— 是改了 prompt 还是换了索引还是切了模型?日志混乱,根本对不上时间。
根因 · 没有 release_id 把「索引 + Prompt + 模型 + Eval」原子绑定 · 任何一个改动都可能引爆事故 · 又找不到怎么回去。

咱们这节聊 4 件事:

- release_id 原子绑定 4 件套 —— 索引 + Prompt + 模型 + Eval
- Release Manifest YAML schema 设计 —— Week 14 治理周的预演
- 秒级回滚机制 —— 不是「重启 docker」· 是「切 pointer 指向旧 release_id」
- Canary + Feature Flag —— LaunchDarkly / Statsig 在 LLM 场景的用法

ATOMIC RELEASE

原子绑定 4 件套 · RAG 上线的最小不可分单元

不是「上线就只能往前走」· 是「能 30 秒切回上周状态」

原理

RAG 服务有 4 个独立可变的的部分 · 索引(切片库)/ Prompt 模板 / 模型版本 / Eval 报告。
任何一个变了 · 答案行为就会变。如果它们各自独立部署 → 排查事故只能「猜」。
解药 · release_id 把 4 件套原子打包 · 上线一起上 / 回滚一起回 / 审计一起查 · 这就是「Release Manifest」。

4 件套绑定 → release_id

索引 (Index)	embedding 模型 / 切片版本 / vector db snapshot_id
Prompt 模板	prompt_id + version (L04 输出)
模型版本	claude-sonnet-4-6 (精确到 build)
Eval 报告	golden set 跑分快照(faithfulness / context_precision)

RELEASE MANIFEST

release.yaml schema · RAG 服务的「DNA」

一份 YAML 决定这次上线的全部行为 · Week 14 治理周的核心制品

```
# releases/rag-2025-06-01.yaml
release_id: rag-2025-06-01-v1.5.0
created_at: "2025-06-01T10:00:00Z"
created_by: zengdan@team
status: active # active / canary / archived / rollbacked
```

```
# 4 件套原子绑定
index:
  snapshot_id: chunks_v8_2025-05-29
  embedding_model: jina-embeddings-v3
  embedding_dim: 1024
  chunker_version: semantic_v2.1
  total_chunks: 84231
```

```
prompt:
  prompt_id: rag_answer
  version: v1.2.3
  git_sha: a3f9d2e
```

```
model:
  primary: claude-sonnet-4-6
  fallback: gpt-4o-2024-08-06
  rerank: rerank-v3.5
  embedder: jina-v3
```

```
eval:
  golden_set: tests/golden/rag_v1.json
  golden_size: 200
  faithfulness: 0.91
  context_precision: 0.87
  citation_accuracy: 0.94
  run_id: eval-20250531-0832
```

```
rollout:
  canary_percent: 5 # 5% 灰度
  rollback_to: rag-2025-05-25-v1.4.8
```

release_id 是「主键」

- 任何答案都能回查到这个 ID
- 客户问「上周怎样」→ 切 release_id

4 件套全字段

- snapshot_id / version / model_id / eval_run_id
- 缺一个就不算原子绑定

rollback_to 是兜底

- 出问题秒切 pointer
- 不需要重新部署服务

status 流转

- canary 5% → active 100% → archived
- 任何状态都可回退

Week 14 治理这一切都基于这份 YAML 展开

SECONDS-LEVEL ROLLBACK

秒级回滚机制 · 不是「重启 docker」· 是「切指针」

事故响应从「6 小时排查 + 重新部署」走到「30 秒切回上版」· ROI 最高的工程动作

```
# services/rag_api/release.py
ACTIVE_RELEASE = "rag-2025-06-01-v1.5.0" # 单一指针 · 改一行 = 切版本
```

```
@app.post("/rag/query")
async def query(request: QueryRequest):
    release = load_release(ACTIVE_RELEASE) # 缓存:O(1) 读
    # release 里指向哪个索引 / 哪个 prompt / 哪个模型 都明确

    chunks = await hybrid_search(
        request.query,
        snapshot_id=release.index.snapshot_id, # 锁定索引版本
    )
    reranked = await rerank(chunks, model=release.model.rerank)
    prompt = render_prompt(
        release.prompt.prompt_id,
        release.prompt.version, # 锁定 prompt 版本
        evidences=reranked,
        query=request.query,
    )
    answer = await llm.call(
        model=release.model.primary, # 锁定 LLM 模型
        prompt=prompt,
    )
    answer.release_id = release.release_id # 关键 · 答案带回 release_id
    return answer
```

```
# 秒级回滚 = 改 ACTIVE_RELEASE 这一个变量 → 重启 / 热更新即可
# 不需要改代码 · 不需要重新构建镜像 · 不需要重新跑 CI
```

关键判断 · 「切 pointer」是 release.yaml 设计的核心目的 · 让事故响应从 6 小时变 30 秒

CANARY + FEATURE FLAG

Canary + Feature Flag · 让 RAG 上线像 deploy 代码

LaunchDarkly / Statsig 在 LLM 场景的用法 · 按 user_id / tenant_id 分流

工具	类型	关键特性 2025	RAG 场景适用
LaunchDarkly	商业 SaaS	企业级 Feature Flag 标杆 / Targeting / Rollout	生产 RAG 灰度 · 按 user_id
Statsig	商业 SaaS	Feature Flag + A/B + Eval 一体 / 免费 tier 慷慨	中小团队 · 自带 A/B
Unleash	开源	Feature Flag OSS / 自托管	自托管首选
OpenFeature	开源标准	CNCF 项目 · 跨厂商 SDK	避免锁定 · 抽象层
自研(release.yaml + 路由)	DIY	简单实用 / 适合小团队	OmniSupport 默认

- RAG 灰度策略 4 种维度 · 常见组合用法:
- 按 tenant 分流 · 5% 租户先用新版 · 出问题不影响所有客户
 - 按 user 哈希 · 20% 用户固定到新版 · 监控 cohort 指标
 - 按 query 类型 · 「FAQ 类」用 v2.0, 「订单类」继续用 v1.5 · 风险隔离
 - 按时段 · 低峰期上新版 · 高峰期回稳定版

INDUSTRY SIGNALS 2025

RAG 版本治理 4 个明显方向 · 2025 已成 To B 合规基线

不是「先上线再治理」· 是「不治理不能上线」· EU AI Act 已经写进合规

LaunchDarkly + LLM Targeting

2024-2025

企业级 Feature Flag 标杆 · Targeting + Rollout 控制。
2025 加 AI Confidence Toggle · 按置信度自动切版本。

MLflow Prompt Registry

2024 末

Databricks 主导 · 把 prompt 当 model 管理。
UC + LakeFS 集成 · 企业级 catalog 友好。

lakeFS for Vectors

2024-2025

「数据 Git」也覆盖 vector index · branch / rollback 索引。
Iceberg 友好 · 适合 Lakehouse 重度团队。

EU AI Act 合规

2024.8 生效

高风险 AI 强制 traceability · release_id 是合规基础。
2025-2026 中欧合规审查必看 · 是 To B 上线刚需。

ANTI-PATTERNS

RAG 版本治理 5 个常见反模式

反模式 1(重启 docker 算回滚)是 2023 时代 · 2025 客户问审计立刻翻车

反模式	具体表现	后果	正确做法
重启 docker 算回滚	回滚 = git revert + 重启	事故响应 1 小时起	release_id 切指针 · 30 秒
4 件套独立部署	索引 / Prompt / 模型各自上线	排查事故只能猜	原子打包 release.yaml
没 release_id 入审计	答案不知道哪个版本	客户问追溯不到	release_id 入 RAGAnswer
Prompt 改了不重测	PM 上线就上,没跑 Eval	指标退化才被发现	CI 强制 Eval 阻断
不灰度直接全切	canary_percent=100 上线	事故概率 5-10x	5% → 25% → 100% 三阶

踩坑提醒

我跟你讲那个 6 小时事故的结局 · 当时根本找不到「上周怎样」的状态 · 最后给客户的回复是「我们升级了 prompt 措辞」——
客户一句:「以后改任何东西都告诉我们」 · 这就是没 release_id 的代价 · 永远的信任损耗。



COST ENGINEERING

Cost Engineering · 让 RAG 服务的月账单不爆炸

我在客户场景算过账 · 按 10 万 query / 月 · 无优化 \$48K → 全套优化 \$9K · 降 81%

优化手段	机制	Cost 降幅	副作用	上线难度
Prompt Caching	Anthropic / OpenAI 缓存 system + skills	60-75%	冷启动 1.25x 写	< 1 天 ★★
Router 便宜模型	简单 query 走 Haiku/mini · 复杂走 Sonnet/4	40-60%	Classifier 一次 cost	2-3 天
Reranker 替代精排	top-50 → reranker top-5 而非塞全 LLM	context 降 75%	reranker 一次 cost	< 1 天 ★★
Embedding 自托管	BGE-M3 / Jina v3 替代 OpenAI emb	embedding 1/5	需 GPU 运维	1 周
Context Pruning	Top-K + Token Budget + LongLLMLingua	50-70%	可能漏关键 chunk	2-3 天
批量化推理	低延迟 query 批量化 (Anthropic batch)	50%	+24h 时延	< 1 天
模型蒸馏	微调小模型替代大模型 (RAG 任务)	80-90%	需训练 pipeline	1-2 月

组合拳 · Prompt Cache + Router + Reranker · 3 件套立刻能省 65-75% · 这是 ROI 最高组合

一条生产级 RAG 服务

L01 · Retrieve

Query + Hybrid
改写 + HyDE + Adaptive
Dense + BM25 + RRF

L02 · Rerank

3 门派 + Multi-Hop
Cross / ColBERT / LLM
Cohere 3.5 / BGE v2

L03 · Generate

受约束 + Context
Citations + Schema
Prompt Cache 省 70%

L04 · Prompt + Eval

Prompt as Code
LLM-Judge + Bad Case
Drift Detection

L05 · Release

原子绑定 + Cost
release.yaml + 秒回滚
三件套省 75%

本周交付物 · 全部 push 至 GitHub omnisupport-copilot 仓库

- pipelines/query/rewriter.py · Query Rewrite + HyDE
- pipelines/retrieve/hybrid.py · Dense + BM25 + RRF
- pipelines/retrieve/multi_hop.py · Iterative Retrieval
- services/rag_api/context_pruning.py · Top-K + Token Budget
- services/rag_api/release.py · release_id 路由
- tests/eval/judge.py · LLM-as-Judge Faithfulness
- monitoring/drift_detector.py · 3 监控点告警

- pipelines/query/router.py · Adaptive RAG (LangGraph)
- pipelines/retrieve/rerank.py · Cohere + ColBERT + LLM
- services/rag_api/models.py · RAGAnswer JSON 契约
- services/rag_api/llm_call.py · Prompt Caching 配套
- prompts/rag_answer/v1.2.3.j2 · Jinja2 模板 + Git
- data/bad_cases/*.jsonl · Bad Case 库 + 4 类标签
- releases/rag-2025-06-01.yaml · Release Manifest